

✓ Facial-Expression Image Classifier — "Depressive" vs "Neutral"

✓ 1. Environment setup

```
# Core libraries
import os, random, time, copy
import numpy as np
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, random_split
from torchvision import datasets, transforms
from torchvision.utils import make_grid

from sklearn.metrics import confusion_matrix, classification_report, ConfusionMatrixDisplay

SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

Using device: cuda

✓ 2. Download the dataset

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("gudiyasingh/emotion-dataset-for-mental-health")
print("Path to dataset files:", path)
```

Using Colab cache for faster access to the 'emotion-dataset-for-mental-health' dataset.
Path to dataset files: /kaggle/input/emotion-dataset-for-mental-health

```
# Inspect the folder structure that kagglehub downloaded
for root, dirs, files in os.walk(path):
    depth = root.replace(path, "").count(os.sep)
    indent = " " * depth
    print(f"{indent}{os.path.basename(root)}/")
    if depth >= 2:
        print(f"{indent} ... {len(files)} files")
```

```
emotion-dataset-for-mental-health/
images/
  validation/
    ... 0 files
  depressive/
    ... 3229 files
  natural/
    ... 3838 files
test/
  ... 0 files
  depressive/
    ... 3229 files
  natural/
    ... 3838 files
train/
  ... 0 files
  depressive/
    ... 3229 files
  natural/
    ... 3838 files
```

```

TRAIN_DIR = os.path.join(path, "images", "train")
VAL_DIR   = os.path.join(path, "images", "validation")
TEST_DIR  = os.path.join(path, "images", "test")

for split_name, split_dir in [("Train", TRAIN_DIR), ("Validation", VAL_DIR), ("Test", TEST_DIR)]:
    if os.path.isdir(split_dir):
        for cls in sorted(os.listdir(split_dir)):
            cls_path = os.path.join(split_dir, cls)
            if os.path.isdir(cls_path):
                print(f"{split_name:12s} | {cls:12s} | {len(os.listdir(cls_path))} images")

```

Train	depressive	3229 images)
Train	natural	3838 images)
Validation	depressive	3229 images)
Validation	natural	3838 images)
Test	depressive	3229 images)
Test	natural	3838 images)

3. Dataset description

Property	Value
Task	Binary image classification
Classes	Depressive , Neutral
Image size	48 × 48
Color mode	Grayscale (1 channel)
Format	PNG/JPG
Split	Train / Validation / Test (provided by the dataset author)
License	CCO: Public Domain

```

def show_samples(directory, n=6):
    classes = sorted([d for d in os.listdir(directory) if os.path.isdir(os.path.join(directory, d))])
    fig, axes = plt.subplots(len(classes), n, figsize=(2*n, 2*len(classes)))
    for row, cls in enumerate(classes):
        cls_dir = os.path.join(directory, cls)
        files = os.listdir(cls_dir)[n:]
        for col, f in enumerate(files):
            img = plt.imread(os.path.join(cls_dir, f))
            ax = axes[row, col] if len(classes) > 1 else axes[col]
            ax.imshow(img, cmap="gray")
            ax.axis("off")
            if col == 0:
                ax.set_ylabel(cls)
    plt.suptitle("Sample images per class")
    plt.tight_layout()
    plt.show()

show_samples(TRAIN_DIR)

```

Sample images per class



4. Preprocessing & data pipeline

```
# First pass with a plain ToTensor transform, just to compute mean/std
basic_transform = transforms.Compose([
    transforms.Grayscale(num_output_channels=1),
    transforms.Resize((48, 48)),
    transforms.ToTensor(),
])

_tmp_train = datasets.ImageFolder(TRAIN_DIR, transform=basic_transform)
_tmp_loader = DataLoader(_tmp_train, batch_size=256, shuffle=False, num_workers=0)

mean = 0.0
std = 0.0
n_samples = 0
for imgs, _ in _tmp_loader:
    b = imgs.size(0)
    imgs = imgs.view(b, -1)
    mean += imgs.mean(1).sum()
    std += imgs.std(1).sum()
    n_samples += b
mean /= n_samples
std /= n_samples
print(f"Computed dataset mean={mean:.4f}, std={std:.4f}")
```

Computed dataset mean=0.5057, std=0.2128

```
# Final transforms used for training / evaluation
train_transform = transforms.Compose([
    transforms.Grayscale(num_output_channels=1),
    transforms.Resize((48, 48)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(10),
    transforms.ToTensor(),
    transforms.Normalize(mean=[mean.item()], std=[std.item()]),
])

eval_transform = transforms.Compose([
    transforms.Grayscale(num_output_channels=1),
    transforms.Resize((48, 48)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[mean.item()], std=[std.item()]),
])

train_ds = datasets.ImageFolder(TRAIN_DIR, transform=train_transform)
val_ds = datasets.ImageFolder(VAL_DIR, transform=eval_transform)
test_ds = datasets.ImageFolder(TEST_DIR, transform=eval_transform)

CLASS_NAMES = train_ds.classes
print("Classes (index -> name):", train_ds.class_to_idx)

BATCH_SIZE = 64
train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True, num_workers=2)
val_loader = DataLoader(val_ds, batch_size=BATCH_SIZE, shuffle=False, num_workers=2)
test_loader = DataLoader(test_ds, batch_size=BATCH_SIZE, shuffle=False, num_workers=2)

print(f"Train: {len(train_ds)} | Val: {len(val_ds)} | Test: {len(test_ds)}")
```

Classes (index -> name): {'depressive': 0, 'natural': 1}
 Train: 7067 | Val: 7067 | Test: 7067

5. CNN architecture

```
class EmotionCNN(nn.Module):
    """Custom CNN for 48x48 grayscale, 2-class emotion image classification."""
    def __init__(self, num_classes=2, dropout1=0.4, dropout2=0.3):
        super().__init__()

        # ---- Feature extractor: 3 conv blocks ----
        self.block1 = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2), # 48 -> 24
```

```

    )
    self.block2 = nn.Sequential(
        nn.Conv2d(32, 64, kernel_size=3, padding=1),
        nn.BatchNorm2d(64),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(2),    # 24 -> 12
    )
    self.block3 = nn.Sequential(
        nn.Conv2d(64, 128, kernel_size=3, padding=1),
        nn.BatchNorm2d(128),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(2),    # 12 -> 6
    )

    # ---- Classifier head ----
    self.flatten = nn.Flatten()
    self.drop1 = nn.Dropout(dropout1)
    self.fc1 = nn.Linear(128 * 6 * 6, 256)
    self.relu_fc = nn.ReLU(inplace=True)
    self.drop2 = nn.Dropout(dropout2)
    self.fc2 = nn.Linear(256, num_classes)

    def forward(self, x):
        x = self.block1(x)
        x = self.block2(x)
        x = self.block3(x)
        x = self.flatten(x)
        x = self.drop1(x)
        x = self.relu_fc(self.fc1(x))
        x = self.drop2(x)
        x = self.fc2(x)          # raw logits; CrossEntropyLoss applies softmax internally
        return x

model = EmotionCNN(num_classes=len(CLASS_NAMES)).to(device)
print(model)

n_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"\nTrainable parameters: {n_params:,}")

```

```

EmotionCNN(
  (block1): Sequential(
    (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (block2): Sequential(
    (0): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (block3): Sequential(
    (0): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (drop1): Dropout(p=0.4, inplace=False)
  (fc1): Linear(in_features=4608, out_features=256, bias=True)
  (relu_fc): ReLU(inplace=True)
  (drop2): Dropout(p=0.3, inplace=False)
  (fc2): Linear(in_features=256, out_features=2, bias=True)
)

Trainable parameters: 1,273,538

```

6. Forward propagation

```

model.eval()
sample_imgs, sample_labels = next(iter(train_loader))
sample_imgs = sample_imgs.to(device)

with torch.no_grad():
    x = sample_imgs

```

```

print("Input                :", tuple(x.shape))

x = model.block1(x)
print("After block1 (conv+pool):", tuple(x.shape))

x = model.block2(x)
print("After block2 (conv+pool):", tuple(x.shape))

x = model.block3(x)
print("After block3 (conv+pool):", tuple(x.shape))

x = model.flatten(x)
print("After flatten         :", tuple(x.shape))

x = model.drop1(x)
x = model.relu_fc(model.fc1(x))
print("After fc1 + ReLU      :", tuple(x.shape))

x = model.drop2(x)
logits = model.fc2(x)
print("Output logits        :", tuple(logits.shape))

probs = F.softmax(logits, dim=1)
print("\nExample softmax probabilities for first 3 images in the batch:")
for i in range(3):
    print(f"  image {i}: {dict(zip(CLASS_NAMES, [round(p, 3) for p in probs[i].tolist()])))}")

```

Input : (64, 1, 48, 48)
 After block1 (conv+pool): (64, 32, 24, 24)
 After block2 (conv+pool): (64, 64, 12, 12)
 After block3 (conv+pool): (64, 128, 6, 6)
 After flatten : (64, 4608)
 After fc1 + ReLU : (64, 256)
 Output logits : (64, 2)

Example softmax probabilities for first 3 images in the batch:
 image 0: {'depressive': 0.504, 'natural': 0.496}
 image 1: {'depressive': 0.495, 'natural': 0.505}
 image 2: {'depressive': 0.492, 'natural': 0.508}

7. Backward propagation

```

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

model.train()
sample_imgs, sample_labels = sample_imgs.to(device), sample_labels.to(device)

# 1. Forward pass
logits = model(sample_imgs)
loss = criterion(logits, sample_labels)
print(f"Loss before this step: {loss.item():.4f}")

# 2. Clear old gradients
optimizer.zero_grad()

# 3. Backward pass: autograd computes d(loss)/d(every parameter)
loss.backward()

print("\nGradient norms (evidence that error is flowing all the way back to layer 1):")
print(f"  fc2.weight   (output layer)   : {model.fc2.weight.grad.norm():.6f}")
print(f"  fc1.weight   (hidden layer)    : {model.fc1.weight.grad.norm():.6f}")
print(f"  block3 conv.weight (deep conv) : {model.block3[0].weight.grad.norm():.6f}")
print(f"  block1 conv.weight (first conv): {model.block1[0].weight.grad.norm():.6f}")

# 4. Update every weight: W <- W - lr * grad (Adam's adaptive version of this rule)
optimizer.step()

with torch.no_grad():
    new_loss = criterion(model(sample_imgs), sample_labels)
    print(f"\nLoss after this single update step: {new_loss.item():.4f}")

```

Loss before this step: 0.7423

Gradient norms (evidence that error is flowing all the way back to layer 1):

```
fc2.weight (output layer) : 1.197909
fc1.weight (hidden layer) : 4.845356
block3 conv.weight (deep conv) : 1.311912
block1 conv.weight (first conv): 0.323614
```

Loss after this single update step: 2.6395

8. Full training loop

```
# Re-initialize a fresh model/optimizer for the real training run
model = EmotionCNN(num_classes=len(CLASS_NAMES)).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode="min", factor=0.5, patience=3)

EPOCHS = 30
best_val_loss = float("inf")
best_state = None
history = {"train_loss": [], "train_acc": [], "val_loss": [], "val_acc": []}

def run_epoch(loader, train=True):
    model.train() if train else model.eval()
    total_loss, correct, total = 0.0, 0, 0
    torch.set_grad_enabled(train)
    for imgs, labels in loader:
        imgs, labels = imgs.to(device), labels.to(device)
        if train:
            optimizer.zero_grad()
            logits = model(imgs)
            loss = criterion(logits, labels)
            if train:
                loss.backward()
                optimizer.step()
            total_loss += loss.item() * imgs.size(0)
            correct += (logits.argmax(1) == labels.sum().item())
            total += imgs.size(0)
    torch.set_grad_enabled(True)
    return total_loss / total, correct / total

start = time.time()
for epoch in range(1, EPOCHS + 1):
    tr_loss, tr_acc = run_epoch(train_loader, train=True)
    val_loss, val_acc = run_epoch(val_loader, train=False)
    scheduler.step(val_loss)

    history["train_loss"].append(tr_loss); history["train_acc"].append(tr_acc)
    history["val_loss"].append(val_loss); history["val_acc"].append(val_acc)

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        best_state = copy.deepcopy(model.state_dict())

    print(f"Epoch {epoch:2d}/{EPOCHS} | train_loss={tr_loss:.4f} train_acc={tr_acc:.4f} "
          f"| val_loss={val_loss:.4f} val_acc={val_acc:.4f}")

print(f"\nTraining finished in {(time.time()-start)/60:.1f} min. Best val_loss={best_val_loss:.4f}")
model.load_state_dict(best_state) # restore best checkpoint
```

```
Epoch 1/30 | train_loss=0.7168 train_acc=0.5695 | val_loss=0.6587 val_acc=0.6083
Epoch 2/30 | train_loss=0.6671 train_acc=0.5926 | val_loss=0.6526 val_acc=0.6155
Epoch 3/30 | train_loss=0.6541 train_acc=0.6015 | val_loss=0.6303 val_acc=0.6471
Epoch 4/30 | train_loss=0.6395 train_acc=0.6208 | val_loss=0.6006 val_acc=0.6621
Epoch 5/30 | train_loss=0.6167 train_acc=0.6491 | val_loss=0.5812 val_acc=0.6846
Epoch 6/30 | train_loss=0.6005 train_acc=0.6649 | val_loss=0.5621 val_acc=0.7006
Epoch 7/30 | train_loss=0.5902 train_acc=0.6727 | val_loss=0.5535 val_acc=0.7061
Epoch 8/30 | train_loss=0.5805 train_acc=0.6857 | val_loss=0.5374 val_acc=0.7160
Epoch 9/30 | train_loss=0.5713 train_acc=0.6931 | val_loss=0.5200 val_acc=0.7388
Epoch 10/30 | train_loss=0.5594 train_acc=0.7047 | val_loss=0.5228 val_acc=0.7210
Epoch 11/30 | train_loss=0.5491 train_acc=0.7119 | val_loss=0.4923 val_acc=0.7556
Epoch 12/30 | train_loss=0.5437 train_acc=0.7144 | val_loss=0.5077 val_acc=0.7562
Epoch 13/30 | train_loss=0.5339 train_acc=0.7238 | val_loss=0.5029 val_acc=0.7521
Epoch 14/30 | train_loss=0.5250 train_acc=0.7248 | val_loss=0.4589 val_acc=0.7753
Epoch 15/30 | train_loss=0.5169 train_acc=0.7379 | val_loss=0.4647 val_acc=0.7682
Epoch 16/30 | train_loss=0.5172 train_acc=0.7427 | val_loss=0.4576 val_acc=0.7759
Epoch 17/30 | train_loss=0.5159 train_acc=0.7336 | val_loss=0.4545 val_acc=0.7808
```

```
Epoch 18/30 | train_loss=0.4986 train_acc=0.7490 | val_loss=0.4418 val_acc=0.7860
Epoch 19/30 | train_loss=0.4962 train_acc=0.7535 | val_loss=0.4434 val_acc=0.7887
Epoch 20/30 | train_loss=0.4952 train_acc=0.7539 | val_loss=0.4217 val_acc=0.8015
Epoch 21/30 | train_loss=0.4833 train_acc=0.7624 | val_loss=0.4252 val_acc=0.7964
Epoch 22/30 | train_loss=0.4828 train_acc=0.7655 | val_loss=0.4899 val_acc=0.7505
Epoch 23/30 | train_loss=0.4841 train_acc=0.7569 | val_loss=0.3988 val_acc=0.8132
Epoch 24/30 | train_loss=0.4835 train_acc=0.7665 | val_loss=0.4463 val_acc=0.7753
Epoch 25/30 | train_loss=0.4697 train_acc=0.7699 | val_loss=0.4211 val_acc=0.8019
Epoch 26/30 | train_loss=0.4648 train_acc=0.7744 | val_loss=0.3823 val_acc=0.8201
Epoch 27/30 | train_loss=0.4538 train_acc=0.7766 | val_loss=0.3981 val_acc=0.8129
Epoch 28/30 | train_loss=0.4646 train_acc=0.7761 | val_loss=0.3853 val_acc=0.8247
Epoch 29/30 | train_loss=0.4458 train_acc=0.7800 | val_loss=0.3688 val_acc=0.8272
Epoch 30/30 | train_loss=0.4484 train_acc=0.7817 | val_loss=0.3914 val_acc=0.8251
```

Training finished in 5.7 min. Best val_loss=0.3688
<All keys matched successfully>

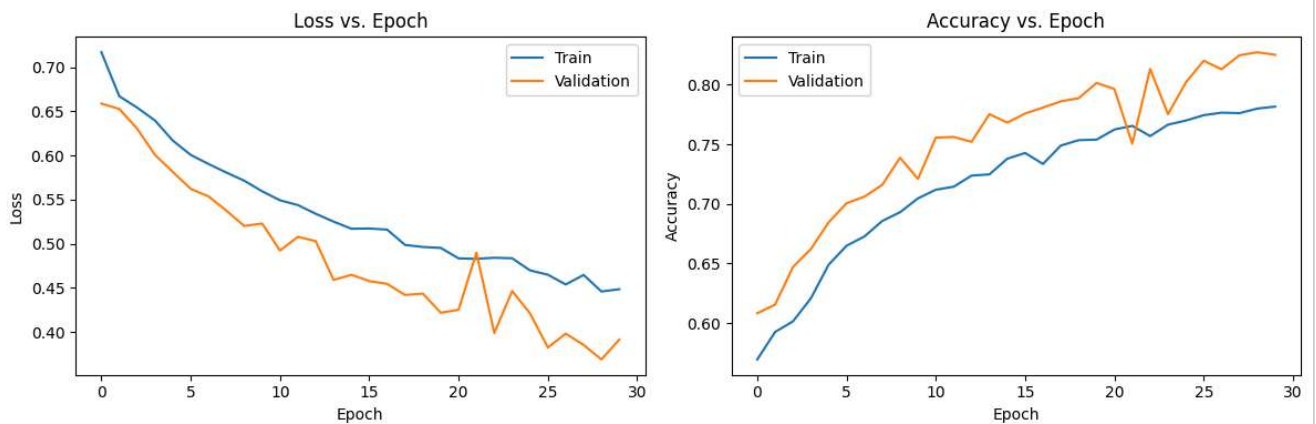
9. Training curves

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(history["train_loss"], label="Train")
axes[0].plot(history["val_loss"], label="Validation")
axes[0].set_title("Loss vs. Epoch"); axes[0].set_xlabel("Epoch"); axes[0].set_ylabel("Loss")
axes[0].legend()

axes[1].plot(history["train_acc"], label="Train")
axes[1].plot(history["val_acc"], label="Validation")
axes[1].set_title("Accuracy vs. Epoch"); axes[1].set_xlabel("Epoch"); axes[1].set_ylabel("Accuracy")
axes[1].legend()

plt.tight_layout()
plt.savefig("training_curves.png", dpi=150)
plt.show()
```



10. Test-set evaluation

```
model.eval()
all_preds, all_labels = [], []
with torch.no_grad():
    for imgs, labels in test_loader:
        imgs = imgs.to(device)
        logits = model(imgs)
        preds = logits.argmax(1).cpu()
        all_preds.extend(preds.tolist())
        all_labels.extend(labels.tolist())

test_acc = np.mean(np.array(all_preds) == np.array(all_labels))
print(f"Test accuracy: {test_acc:.4f}\n")
print(classification_report(all_labels, all_preds, target_names=CLASS_NAMES))
```

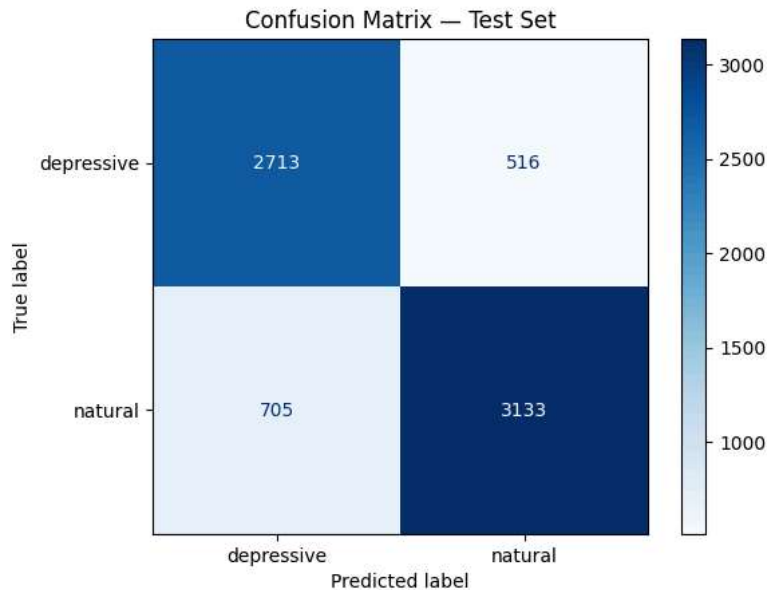
```

cm = confusion_matrix(all_labels, all_preds)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=CLASS_NAMES)
disp.plot(cmap="Blues")
plt.title("Confusion Matrix - Test Set")
plt.savefig("confusion_matrix.png", dpi=150)
plt.show()

```

Test accuracy: 0.8272

	precision	recall	f1-score	support
depressive	0.79	0.84	0.82	3229
natural	0.86	0.82	0.84	3838
accuracy			0.83	7067
macro avg	0.83	0.83	0.83	7067
weighted avg	0.83	0.83	0.83	7067



```

# Save the trained model for later reuse (e.g. in the upload-and-predict section)
torch.save({
    "model_state_dict": model.state_dict(),
    "class_to_idx": train_ds.class_to_idx,
    "mean": mean.item(),
    "std": std.item(),
}, "emotion_cnn.pt")
print("Saved model to emotion_cnn.pt")

```

Saved model to emotion_cnn.pt

```

from google.colab import drive
import os
import shutil

# 1. Mount Google Drive
drive.mount('/content/drive')

# 2. Define the path in Google Drive where you want to save your model
# You can change 'my_colab_models' to any folder name you prefer in your Drive
drive_model_dir = '/content/drive/My Drive/my_colab_models'
os.makedirs(drive_model_dir, exist_ok=True)

# 3. Define the source path of the saved model (from cell Mx_-7LFPQ7NJ)
source_model_path = 'emotion_cnn.pt'

# 4. Define the destination path in Google Drive
destination_model_path = os.path.join(drive_model_dir, source_model_path)

# 5. Copy the model file to Google Drive
try:
    shutil.copy(source_model_path, destination_model_path)

```

```

print(f"Model '{source_model_path}' successfully copied to '{destination_model_path}'")
except FileNotFoundError:
    print(f"Error: '{source_model_path}' not found. Please ensure cell 'Mx_-7LFPQ7NJ' was executed to save the model before att
except Exception as e:
    print(f"An error occurred while copying the model to Google Drive: {e}")

print("\n--- How to load this model from Google Drive in a new session ---")
print("1. Mount Google Drive using: `from google.colab import drive; drive.mount('/content/drive')`")
print(f"2. Update the `torch.load` line in your prediction cell (e.g., cell Y1qS9lpEiXyh) to point to:")
print(f"    `saved_model_data = torch.load('{destination_model_path}', map_location=torch.device('cpu'))`")

```

```

Mounted at /content/drive
Model 'emotion_cnn.pt' successfully copied to '/content/drive/My Drive/my_colab_models/emotion_cnn.pt'

```

```

--- How to load this model from Google Drive in a new session ---
1. Mount Google Drive using: `from google.colab import drive; drive.mount('/content/drive')`
2. Update the `torch.load` line in your prediction cell (e.g., cell Y1qS9lpEiXyh) to point to:
    `saved_model_data = torch.load('/content/drive/My Drive/my_colab_models/emotion_cnn.pt', map_location=torch.device('cpu'))`

```

11. Image Result Prediction

```

import torch
import torch.nn as nn
import torch.nn.functional as F

class EmotionCNN(nn.Module):
    """Custom CNN for 48x48 grayscale, 2-class emotion image classification."""
    def __init__(self, num_classes=2, dropout1=0.4, dropout2=0.3):
        super().__init__()

        # ---- Feature extractor: 3 conv blocks ----
        self.block1 = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2), # 48 -> 24
        )
        self.block2 = nn.Sequential(
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2), # 24 -> 12
        )
        self.block3 = nn.Sequential(
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2), # 12 -> 6
        )

        # ---- Classifier head ----
        self.flatten = nn.Flatten()
        self.drop1 = nn.Dropout(dropout1)
        self.fc1 = nn.Linear(128 * 6 * 6, 256)
        self.relu_fc = nn.ReLU(inplace=True)
        self.drop2 = nn.Dropout(dropout2)
        self.fc2 = nn.Linear(256, num_classes)

    def forward(self, x):
        x = self.block1(x)
        x = self.block2(x)
        x = self.block3(x)
        x = self.flatten(x)
        x = self.drop1(x)
        x = self.relu_fc(self.fc1(x))
        x = self.drop2(x)
        x = self.fc2(x) # raw logits; CrossEntropyLoss applies softmax internally
        return x

```

```

import ipywidgets as widgets
from IPython.display import display, clear_output
from PIL import Image
import io
from torchvision import transforms
import torch

```

```

from google.colab import drive # Import drive
import matplotlib.pyplot as plt # Import matplotlib.pyplot

# Define device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# --- Mount Google Drive ---
drive.mount('/content/drive')

# Define the path to the model in Google Drive
drive_model_path = '/content/drive/My Drive/my_colab_models/emotion_cnn.pt'

# --- Load the saved model and its parameters ---
try:
    # Load the model from Google Drive
    saved_model_data = torch.load(drive_model_path, map_location=torch.device('cpu'))
    mean = saved_model_data['mean']
    std = saved_model_data['std']
    CLASS_NAMES = [name for name, idx in sorted(saved_model_data['class_to_idx'].items(), key=lambda item: item[1])]

    # Re-initialize the model structure and load the saved weights
    # EmotionCNN class is expected to be defined in a previous cell (Gw-pmUVCQ7NG)
    model = EmotionCNN(num_classes=len(CLASS_NAMES))
    model.load_state_dict(saved_model_data['model_state_dict'])
    model.to(device) # 'device' is expected to be defined in a previous cell (1jAnu3EfQ7M9)
    print(f"Model, mean, std, and class names loaded successfully from {drive_model_path}")

    infer_transform = transforms.Compose([
        transforms.Grayscale(num_output_channels=1),
        transforms.Resize((48, 48)),
        transforms.ToTensor(),
        transforms.Normalize(mean=[mean], std=[std]),
    ])

    # 'F' (torch.nn.functional) and 'plt' (matplotlib.pyplot) are expected to be imported in a previous cell (1jAnu3EfQ7M9)
    def predict_image(pil_img, show=True):
        model.eval()
        tensor = infer_transform(pil_img).unsqueeze(0).to(device)
        with torch.no_grad():
            logits = model(tensor)
            probs = F.softmax(logits, dim=1)[0].cpu().numpy()
        pred_idx = int(probs.argmax())
        pred_class = CLASS_NAMES[pred_idx]
        if show:
            plt.figure(figsize=(3, 3))
            plt.imshow(pil_img.convert("L"), cmap="gray")
            plt.axis("off")
            plt.title(f"Predicted: {pred_class} ({probs[pred_idx]*100:.1f}%)" # Removed Chinese characters here
            plt.show()
        return pred_class, probs

    # --- Setup the file uploader widget ---
    uploader = widgets.FileUpload(accept="image/*", multiple=False)
    out = widgets.Output()

    def on_upload_change(change):
        with out:
            clear_output()
            # Handle potential differences in how uploader.value is structured
            uploaded_files = uploader.value
            if isinstance(uploaded_files, dict):
                # For a single file upload, value might be a dict {filename: file_info}
                file_info = next(iter(uploaded_files.values()))
                content = file_info["content"]
            elif isinstance(uploaded_files, list):
                # For multiple files or if it returns a list of FileUpload objects
                file_info = uploaded_files[0] # Assuming single file for now
                content = file_info["content"]
            else:
                print("Unsupported upload format.")
                return

            pil_img = Image.open(io.BytesIO(content))
            predict_image(pil_img)

    uploader.observe(on_upload_change, names="value")
    display(widgets.VBox([widgets.Label("Upload a face image:"), uploader, out]))

```

```
except FileNotFoundError:
    print(f"Error: '{drive_model_path}' not found. Please ensure Google Drive is mounted and the model file exists at this path.
except Exception as e:
    print(f"An error occurred during model loading or prediction setup: {e}")
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
Model, mean, std, and class names loaded successfully from /content/drive/My Drive/my_colab_models/emotion_cnn.pt

Upload a face image:

Upload (4)

Predicted: natural (99.4%)



Start coding or [generate](#) with AI.